



MME 321 Lab 3

Kyle Buttray

2/17/26

1.0 Introduction

The purpose of this lab was to model a coupled translational-rotational spring-mass-damper system. This allows the student to gain a broader understanding of how to model complex mechanical systems as equations governed by the laws of physics. During the course of this lab, MATLAB and Simulink were used to compute and model the translational-rotational system in order to generate graphs of the system's dynamic inputs.

2.0 System Description

The coupled translational-rotational spring-mass-damper system is a physical system governed by the following parameters,

- $J = 1.0$
- $r = 0.4$
- $m = 0.5$
- $k = 1.0$
- $b = 0.25$
- $kr = 0.5$
- $br = 0.25$
- $F(t) = 1$

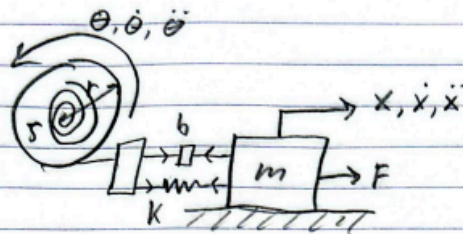
The system consists of a disk with moment of inertia J and radius r rotating connected to both a spring of force k with a constant of kr , and a damper of force b and constant of br . The other ends of the spring and damper are connected with a mass m that is being dragged away from the springs by a constant force $F(t)$. Due to the direction of motion of the mass m , the springs are in tension and the disk is being rotated counterclockwise.

3.0 Derivation of Equations of Motion

The derivation of the equations used to model the dynamic response of this mechanical system is shown in Fig.1. The system includes the variables: F_b , F_k , T_b , and T_k which are the damper force, spring force, rotational damper torque, and rotational spring torque respectively. One expression solves for x double dot and the other solves for θ double dot, which describes the translational and rotational motion of the system entirely.

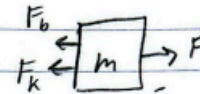
Kyle Buttay
2/17/26

MME 321 Lab 3



Linear motion assume $x > r\theta$

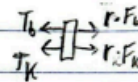
$$F_b = b(\dot{x} - r\dot{\theta}) \quad F_k = k(x - r\theta)$$



$$\sum F = m\ddot{x} \quad m\ddot{x} = F - F_b - F_k \Rightarrow \boxed{\ddot{x} = \frac{1}{m}(F - b(\dot{x} - r\dot{\theta}) - k(x - r\theta))} \quad \text{eq 1}$$

Rotational motion

$$T_b = b_r\dot{\theta} \quad T_k = k_r\theta$$



$$\sum T = J\ddot{\theta} \quad J\ddot{\theta} = r \cdot F_b + r \cdot F_k - T_b - T_k \Rightarrow \boxed{\ddot{\theta} = \frac{1}{J}(r(k(x - r\theta) + b(\dot{x} - r\dot{\theta}))) - b_r\dot{\theta} - k_r\theta} \quad \text{eq 2}$$

Fig.1

4.0 Simulink Modeling

Fig.2 Shows the variable workspace that was used for this Simulink model. These include the values of the parameter used by the system to calculate results.

Workspace		
Name ▲	Value	
b	0.2500	
bt	0.2500	
f	@(t,y)[y(2);(1/m)*(F-...	
F	1	
J	1	
k	1	
kt	0.5000	
m	0.5000	
out	1x1 SimulationOutput	
r	0.4000	
t	101x1 double	
y	101x4 double	

Fig.2

The model shown in Fig.3 shows the system itself and the path that Simulink uses to generate results from the second order differential equations and the parameters provided to solve them.

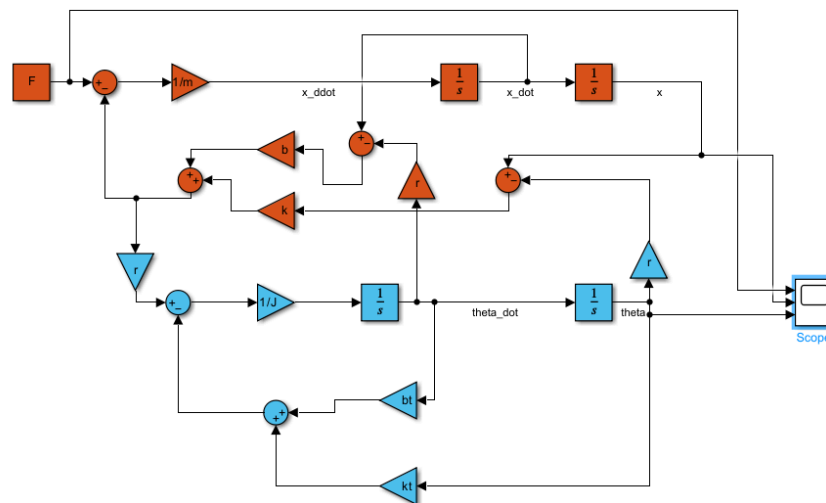


Fig.3

The results of force, distance (x), velocity (x_dot), and rotational distance (theta) of the system over a 10 second span are displayed in Fig.4. These results are dynamic responses (not linear) and would be difficult to graph without computational software.

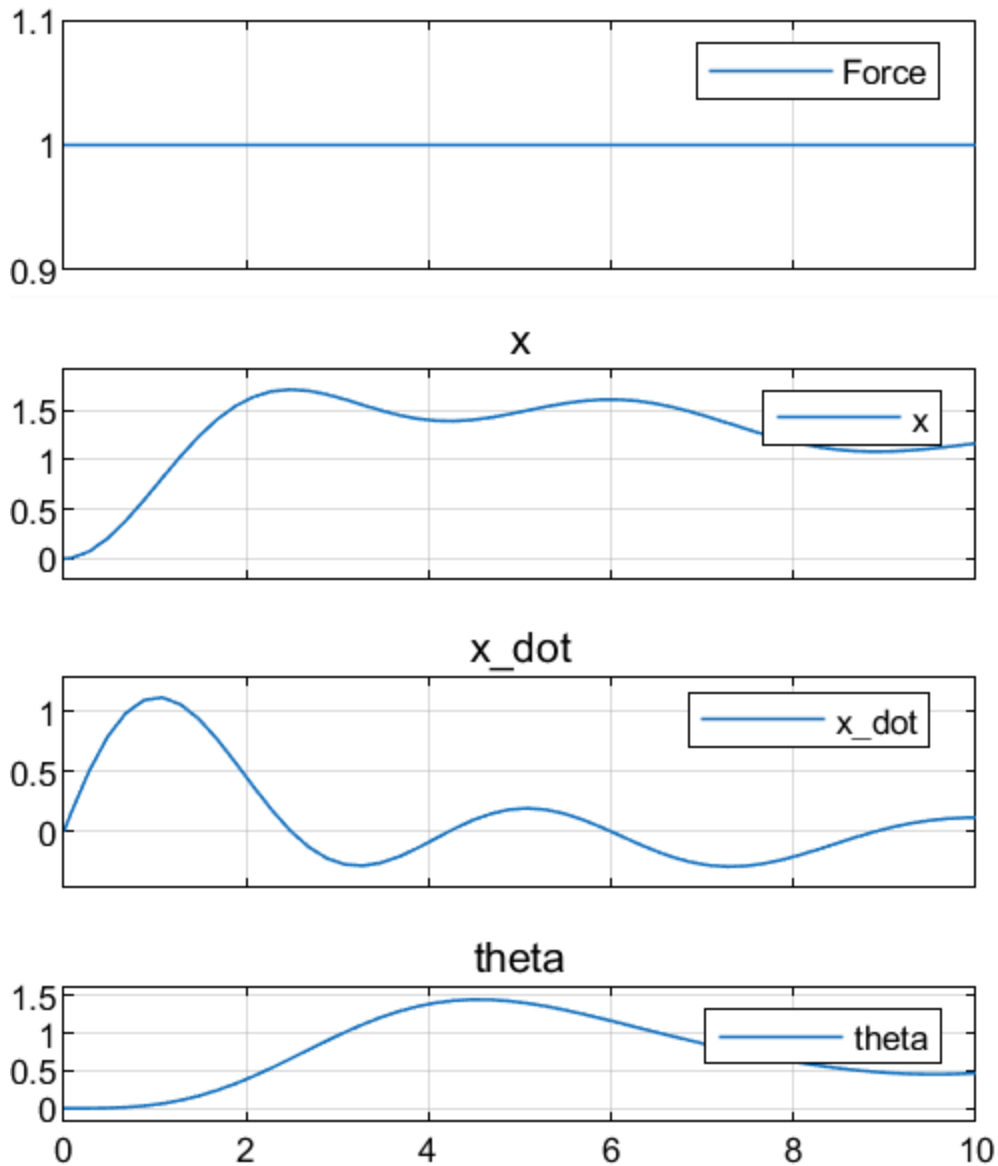


Fig.4

5.0 MATLAB Modeling

Fig.5 shows the code run by MATLAB to model the system outputs as a function of time. This includes the system parameters and a function used to describe the motion of the system.

```
J = 1.0;
r = 0.4;
m = 0.5;
```

```

k = 1.0;
b = 0.25;
kt = 0.5;
bt = 0.25;
F = 1;

```

```

f=@(t,y) ...
    [y(2); ...
    (1/m) * (F-b*(y(2)-r*y(4))-k*(y(1)-r*y(3))); ...
    y(4); ...
    (1/J)*(r*(k*(y(1)-r*y(3)))+b*(y(2)-r*(y(4)))-bt*y(4)-kt*y(3))];

```

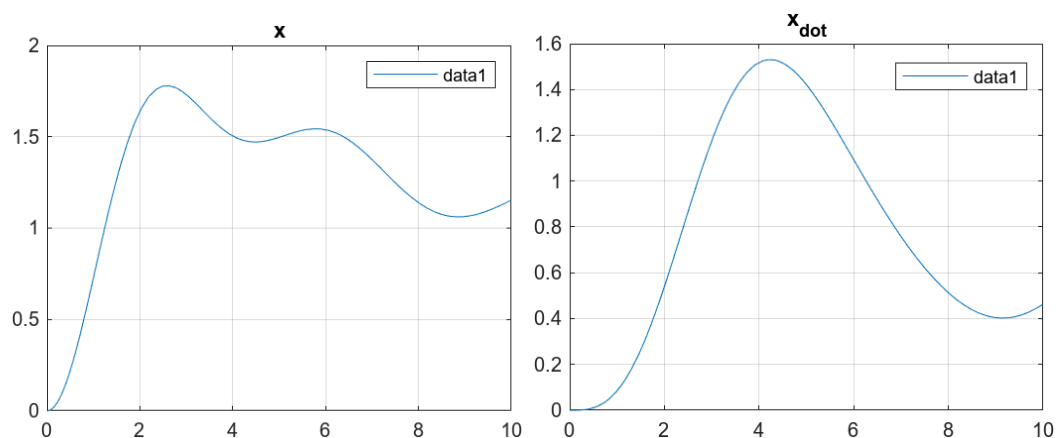
```

figure()
[t,y]= ode45(f, [0 10], [0, 0, 0, 0]);
plot(t,y(:,1)); title('x'); grid on;
plot(t,y(:,3)); title('x_d_o_t'); grid on;
plot(t,y(:,2)); title('theta'); grid on;
plot(t,y(:,4)); title('theta_d_o_t'); grid on;

```

Fig.5

The outputs of the MATLAB function were graphed as functions of time so that we can see the system's dynamic response over a time interval of 10 seconds. This is shown in Fig.6.



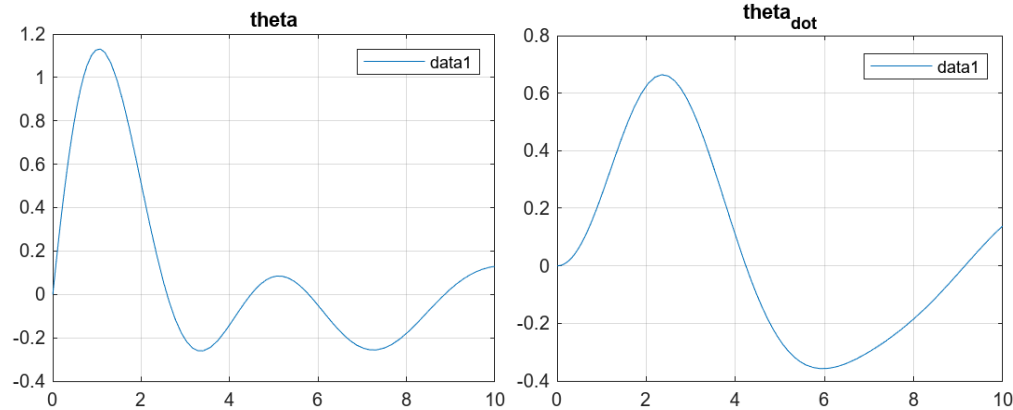


Fig.6

At first glance, it looks like MATLAB and Simulink gave different system responses. However, the graphs created by matlab simply have a larger height than those of Simulink. Additionally it looks like the graph for theta might have been truncated by Simulink, although I am not sure how this is possible as they were given the same span of 10 seconds.

6.0 Conclusion

Overall, dynamic systems are very difficult to simulate manually. It is especially difficult with a problem such as this that involves both rotational and translational motion due to having separate springs and dampers for both individual subsystems. Tools like Simulink or MATLAB make finding the responses of these systems much easier and faster, especially on a large scale. However, it is important to recognize that there may be subtle differences between graphs generated by Simulink and MATLAB due to the nature of different aspect ratios of the generated graphs. While this does not create a change in the data between the two programs, it may be concerning to others who are not aware of this difference.